

Git Assignment 2

Open Source Contribution Simulation

Contributing to a popular open-source project on GitLab: fork, issue, merge request, review, squash and release notes

Field	Details
Student name	Mohit
Institution	National Institute of Information Technology
Course	Java Full stack
Instructor	Mr. Praveen
Submission date	October 03, 2026

Table of Contents

1. Executive Summary	3
2. Business Scenario and Objectives	3
2.1 Scenario	3
2.2 Objectives	3
3. Stakeholders	3
4. Current Challenges and Risks	4
5. Assumptions and Constraints	4
5.1 Given Requirements (from the assignment)	4
5.2 Assumptions (made by the author)	5
5.3 Constraints	5
6. Proposed Solution (Step-by-Step Approach)	5
6.1 Choose the Project and Read the Rules	6
6.2 Fork and Set Up the Repository	6
6.3 Raise an Issue and Agree on the Change	7
6.4 Create a Feature Branch and Implement the Change	8
6.5 Open a Merge Request from the Fork	8
6.6 Address Review Feedback	9
6.7 Squash Commits into a Clean History	9
6.8 Merge, Clean Up and Publish Release Notes	10
7. Presentation and Justification	11
Why this approach is justified	11
8. Implementation Plan	12
Tools and preparation	12
9. Measurable Success Criteria	12
10. Recommendations and Lessons Learned	13
10.1 Recommendations	13
10.2 Lessons Learned	13

1. Executive Summary

Problem. Students act as contributors to a popular open-source project. They are outsiders: they do not have write access, do not know the codebase, and must follow the maintainers' rules. Careless contributions waste maintainers' time and are often rejected.

Solution. A step-by-step contribution workflow that follows professional open-source practice: read the project rules, **fork** the repository, raise an **issue** and agree on the change, build it on a **feature branch**, submit a **merge request (MR)** from the fork, answer **review feedback**, **squash** commits into a clean history, and prepare **release notes**.

Why it works. Maintainers get small, discussed, tested and well-described changes, and the contributor learns the habits that real projects expect: communication first, code second, and respect for the project's process.

Note on terms. The assignment brief uses GitHub words. This report is written for GitLab, where a pull request is called a **merge request** and the first step is also called a **fork**.

2. Business Scenario and Objectives

2.1 Scenario

Students act as contributors to a popular open-source project. They must fork repositories, create feature branches, raise issues, implement enhancements, submit merge requests, address review feedback, squash commits and publish release notes. The objective is to understand professional collaboration practices on a Git hosting platform.

In this report the exercise is run on a **practice project** that behaves like a real open-source project (for example, a small command-line tool called awesome-cli, with an example group awesome-org). Names are illustrative.

2.2 Objectives

#	Objective	Why it matters
O1	Understand and follow the project's contribution rules	Contributions that ignore the rules are rejected
O2	Work safely through a fork without write access	This is how outsiders contribute everywhere
O3	Discuss changes in an issue before coding	Avoids wasted work and duplicate effort
O4	Submit small, clear, tested merge requests	Easy for maintainers to review and merge
O5	Respond to review feedback professionally	Reviews are a conversation, not a judgement
O6	Keep a clean history and document the change in release notes	Makes the project easy to maintain and understand

3. Stakeholders

A stakeholder is anyone affected by, or responsible for, the outcome of the contribution.

Stakeholder	Role	What they need
Student contributor	Proposes and builds the change	Clear rules, quick feedback and a fair review
Maintainers	Own the project and approve merges	Small, well-explained, tested merge requests

Stakeholder	Role	What they need
Reviewers	Read the code and give feedback	Clean code, clear description, respectful discussion
Other contributors	Work on other issues at the same time	No duplicate work or conflicting changes
Users of the project	Use the released software	Stable releases and honest release notes
Instructor	Assesses the simulation	Evidence of each step and good practice
Project community	Follows the code of conduct	A welcoming, respectful environment
CI system (runners)	Runs automatic checks	Tests and linting that pass

4. Current Challenges and Risks

#	Challenge	Impact	Risk
C1	Not reading CONTRIBUTING.md and the code of conduct	Merge request rejected or ignored	High
C2	Coding before discussing in an issue	Work wasted if maintainers disagree	High
C3	Fork falls behind the original project (upstream)	Conflicts and outdated code	Medium
C4	Large merge requests that change many things	Slow review or rejection	High
C5	Failing tests or lint checks	Merge blocked until fixed	Medium
C6	Slow or unclear review feedback	Contribution stalls	Medium
C7	Messy history (many "fix" commits)	Hard to read, revert or release	Medium
C8	Misusing force push or rewriting shared history	Lost work for others	High
C9	Licence or sign-off rules ignored (for example DCO)	Contribution cannot legally be accepted	High
C10	Tone problems in discussions	Conflict and damaged reputation	Medium
C11	Secrets or personal data committed by mistake	Public exposure that cannot be undone	High

Risk level is a judgement based on the likely effect on the contribution and the contributor's reputation. It is not measured data.

5. Assumptions and Constraints

This section separates what the assignment **requires** from what this report **assumes**.

5.1 Given Requirements (from the assignment)

#	Requirement
R1	Students act as contributors to a popular open-source project
R2	Fork repositories and create feature branches
R3	Raise issues and implement enhancements
R4	Submit merge (pull) requests and address review feedback
R5	Squash commits and publish release notes

#	Requirement
R6	Understand professional collaboration practices
R7	Identify stakeholders and objectives; analyse challenges and risks
R8	Create a step-by-step solution; document assumptions and constraints
R9	Prepare presentation and justification; define measurable success criteria
R10	Deliver a detailed report, workflow diagrams, recommendations, lessons learned and an implementation plan

5.2 Assumptions (made by the author)

#	Assumption	Effect if wrong
A1	Project, group, issue and MR numbers are illustrative examples	Real names differ; the method stays the same
A2	GitLab (gitlab.com) is the platform, as in the other assignments	Another platform uses different names for the same steps
A3	The exercise runs on a practice project, with the instructor and volunteers acting as maintainers	On a real project, response times and rules will vary
A4	The project has CONTRIBUTING.md, a code of conduct, a licence, tests and CI	Missing files mean asking the maintainers in an issue first
A5	The contributor has Git installed and basic Git knowledge	A short setup session would be needed
A6	The change is small (about one to three days of work)	A larger change must be split into several merge requests
A7	Targets in Section 9 are illustrative because no baseline data is given	Re-base targets on real data

5.3 Constraints

- **No write access:** contributors can only change their own fork and propose changes.
- **Maintainers decide:** a good merge request can still be rejected or changed.
- **Respect for volunteers:** maintainers' time is limited; avoid spam and large unrequested changes.
- **Licence and rules:** contributions must follow the project's licence, code of conduct and any sign-off rule.
- **Scope:** this is a simulation; no real project is disturbed without permission.

6. Proposed Solution (Step-by-Step Approach)

The solution has eight steps, taken in order. Together they take a contributor from choosing a project to a documented, released change.

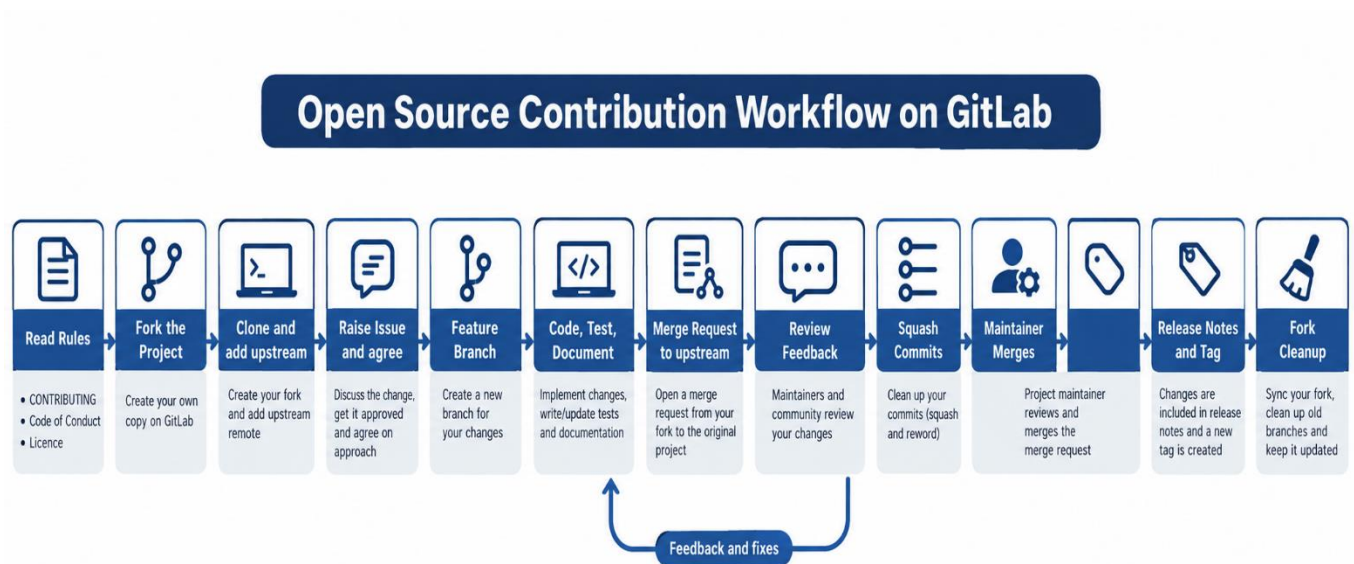


Diagram 1: END-TO-END OPEN SOURCE CONTRIBUTION WORKFLOW

6.1 Choose the Project and Read the Rules

Simple explanation: before touching any code, learn how the project works and what it expects from contributors.

What to read	What to look for
README.md	What the project does and how to run it
CONTRIBUTING.md	Branch names, commit style, test rules, how to submit a merge request
CODE_OF_CONDUCT.md	Expected behaviour in discussions
LICENSE	The terms your contribution is released under
Issue list and labels	Open issues, "good first issue" and "help wanted" labels
Closed merge requests	What was accepted or rejected, and the review style
CI configuration (.gitlab-ci.yml)	Which checks must pass

How to choose: pick an active project (recent commits and replies), with a clear guide for contributors and issues labelled for beginners.

6.2 Fork and Set Up the Repository

Simple explanation: a **fork** is your own copy of the project on GitLab. You have full control of the fork. You then **clone** it to your computer and add a second remote called **upstream** that points to the original project, so you can stay up to date.

Name	Points to	Used for
upstream	The original project (awesome-org/awesome-cli)	Download the latest changes (read only for you)
origin	Your fork (your-username/awesome-cli)	Push your branches
Local copy	Your computer	Write and test code

1. Fork in the GitLab web page, then clone YOUR fork

```
git clone https://gitlab.com/<your-username>/awesome-cli.git
cd awesome-cli

# 2. Add the original project as "upstream"
git remote add upstream https://gitlab.com/awesome-org/awesome-cli.git
git remote -v

# 3. Keep your fork up to date (do this before each new branch)
git fetch upstream
git checkout main
git merge --ff-only upstream/main
git push origin main
```

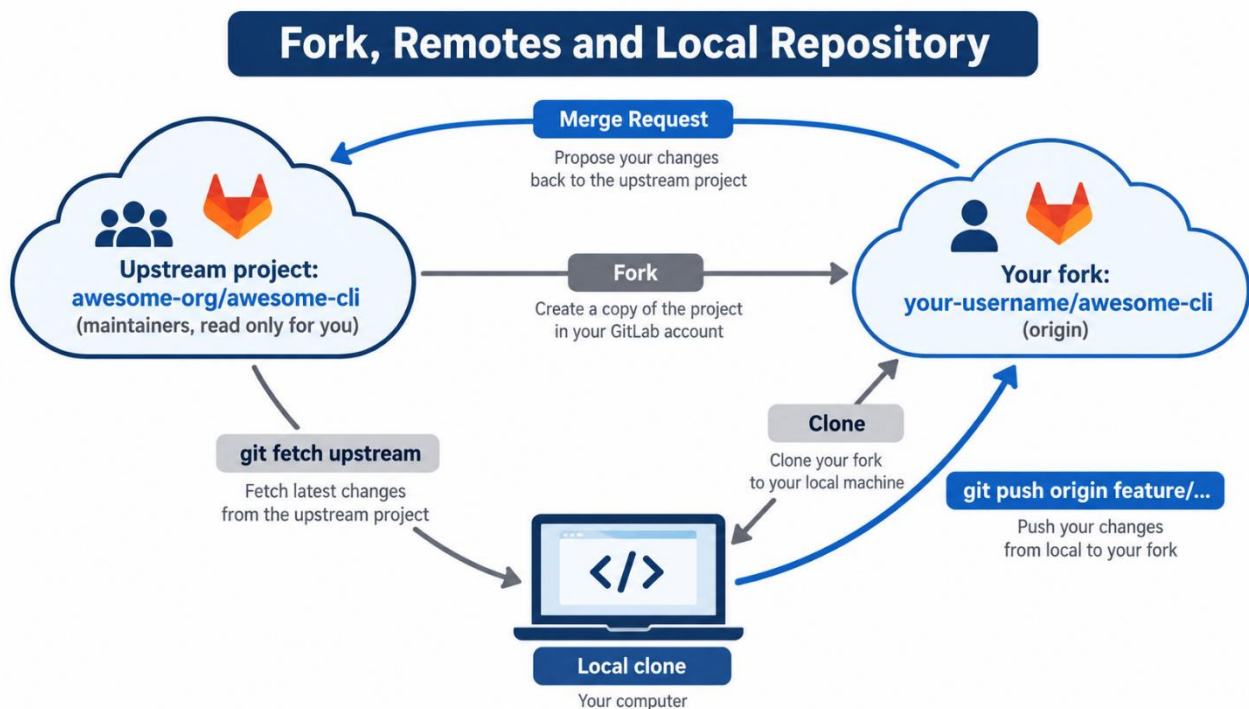


Diagram 2: FORK, REMOTES AND LOCAL REPOSITORY

6.3 Raise an Issue and Agree on the Change

Simple explanation: an **issue** describes the problem or idea. Discussing it first tells you whether maintainers want the change and how they would like it done.

1. **Search** open and closed issues to be sure the idea is new.
2. **Create the issue** using the project's template; describe the problem, the proposed solution and alternatives.
3. **Say you want to work on it** and wait for a maintainer to agree or assign it.
4. **Note the issue number** (for example #214); it is used in the branch name, commits and merge request.

Example issue (illustrative):

```
Title: Add --dry-run flag to the export command

## Summary
The export command writes files immediately, with no way to preview.

## Proposed change
Add a --dry-run flag that lists the files that would be written.

## Alternatives considered
```

A separate "preview" command (more code to maintain).

I would like to work on this.

6.4 Create a Feature Branch and Implement the Change

Simple explanation: never work on the main branch of your fork. A **feature branch** keeps each change separate and easy to discard or update.

```
git checkout -b feature/214-dry-run-flag
# write code, add tests, update documentation
git add .
git commit -m "feat(export): add --dry-run flag (#214)"
git push -u origin feature/214-dry-run-flag
```

Good practice	Detail
One issue, one branch	Keep the change focused; do not mix unrelated fixes
Follow the code style	Run the linter and formatter the project uses
Add or update tests	New behaviour needs a test; all existing tests must pass
Update documentation	README or help text if behaviour changes
Clear commit messages	Type, area, short summary and issue number
Sign-off if required	Use git commit -s when the project asks for a Developer Certificate of Origin
No secrets	Check the diff before every commit

6.5 Open a Merge Request from the Fork

Simple explanation: a **merge request** from your fork asks the maintainers to bring your branch into the original project.

1. Push the branch to your fork and choose **Create merge request**.
2. Set the **target** to the upstream project and its default branch.
3. Start as a **Draft** if the work is not finished.
4. Write a clear title and description; add **"Closes #214"** so the issue links to the change.
5. Fill the project's merge request template and tick the checklist.
6. Check that the **pipeline passes** and fix any failure.
7. Allow commits from members who can merge to the target branch, if the project asks for it.

Merge request description (example):

```
## What
Adds a --dry-run flag to the export command.

## Why
Users want to preview output before writing files. Closes #214.

## How tested
- Added unit tests for dry-run output
- Ran the full test suite locally: all pass

## Checklist
- [x] Tests added   - [x] Docs updated   - [x] Linter passes
```

A good merge request is **small** (aim for under about 300 changed lines), has one purpose, and explains the reason, not only the code.

6.6 Address Review Feedback

Simple explanation: reviewers leave comments in **threads** on specific lines. Some are required changes, some are questions and some are suggestions. Treat feedback as help, not criticism.

Situation	What the contributor does
Reviewer asks for a change	Make the change, push to the same branch (the MR updates by itself), then reply "Done" and resolve the thread
Reviewer gives a suggested change	Read it, then use "Apply suggestion" if you agree
Reviewer asks a question	Answer clearly; update the code or comments if the reader was confused
You disagree	Explain your reason politely with facts; accept the maintainer's final decision
Pipeline fails	Open the job log, fix the cause and push again
No reply for several days	Add a polite reminder after about a week; do not repeatedly ping people
Upstream changed (conflict)	Fetch upstream, rebase your branch, resolve conflicts, test and push

Communication tips: thank reviewers, be specific, keep replies short, and stay calm. All discussion follows the code of conduct.

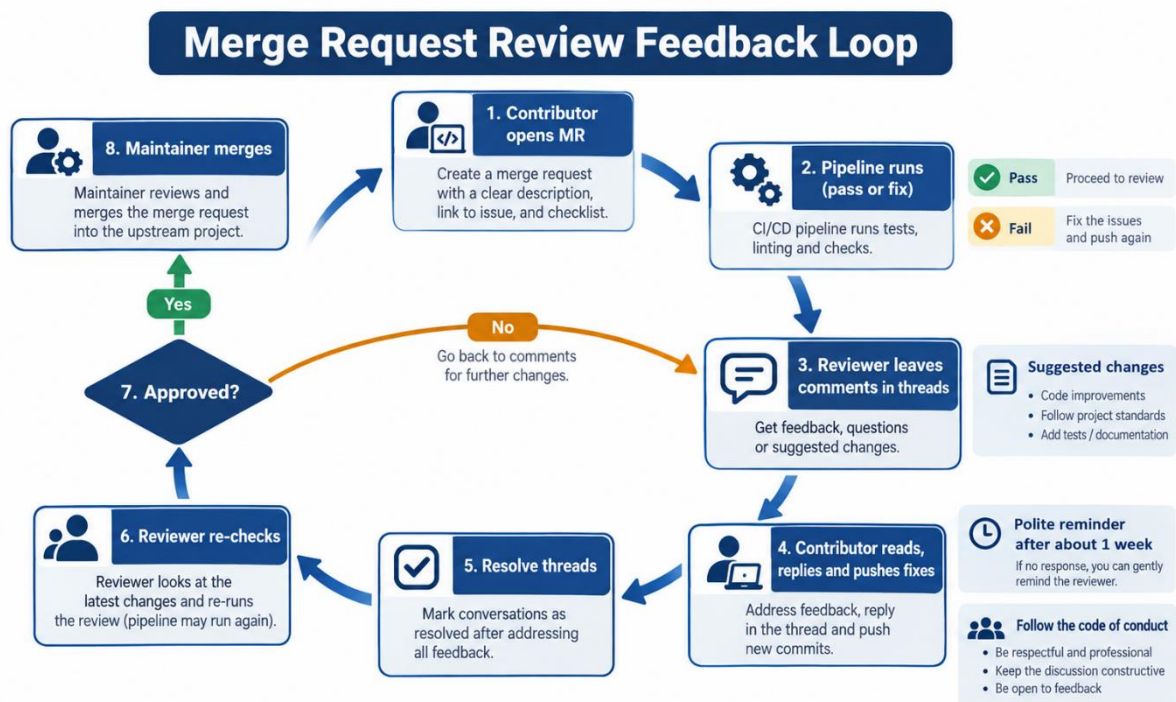


Diagram 3: MERGE REQUEST REVIEW FEEDBACK LOOP

6.7 Squash Commits into a Clean History

Simple explanation: while working you may make many small commits such as "fix typo" or "address review". **Squashing** combines them into one or a few meaningful commits, so the project history stays readable.

Method	Who does it	Notes
Interactive rebase	Contributor, before merge	Gives full control of commit messages
Squash on merge (checkbox on the MR)	Maintainer, at merge time	One commit is created from the whole MR

Method	Who does it	Notes
Fixup commits	Contributor, while working	git commit --fixup, then autosquash at the end

```
git fetch upstream
git rebase -i upstream/main
# in the editor: keep the first commit as "pick",
# change the others to "squash" (or "fixup"), save, then write one clear message

# your branch history changed, so update the fork branch safely:
git push --force-with-lease origin feature/214-dry-run-flag
```

Safety rule: rewrite history (rebase and force push) **only on your own feature branch**, never on main or on a branch other people use. The option `--force-with-lease` refuses to overwrite work you have not seen. Do the squash after the reviewers have finished, or follow the maintainer's preference, because rewriting history during review can make it harder to see what changed.

Squashing Commits: Before and After

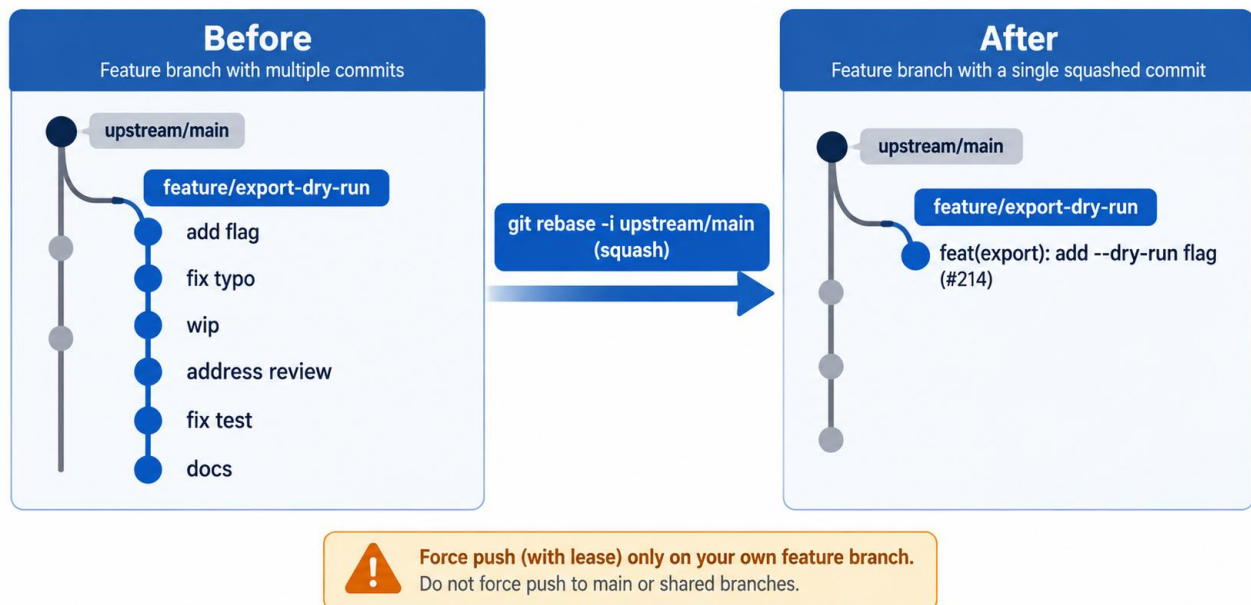


Diagram 4: SQUASHING COMMITS: BEFORE AND AFTER

6.8 Merge, Clean Up and Publish Release Notes

1. **Maintainer merges** the merge request. Issue #214 closes automatically.
2. **Contributor cleans up:** sync the fork with upstream and delete the finished branch locally and on the fork.
3. **Release notes entry:** describe the change in the project's changelog (for example CHANGELOG.md) or in the MR, using plain words for users.
4. **Maintainers tag a release** (for example v1.4.0) and publish a **GitLab Release** that includes the notes.
5. **Contributor checks** that the change and credit appear in the release.

```
git checkout main && git fetch upstream && git merge --ff-only upstream/main
git push origin main
git branch -d feature/214-dry-run-flag
git push origin --delete feature/214-dry-run-flag
```

Release notes example (illustrative):

```
## [1.4.0] - 2026-10-20

### Added
- `export --dry-run` flag shows which files would be written, without writing them (#214, !231)

### Fixed
- Export no longer fails on file names with spaces (#198, !225)
```

Good release notes group changes (Added, Changed, Fixed), use user-friendly language, and link to the issue and merge request numbers. Many projects use semantic versioning: a new feature raises the MINOR number and a bug fix raises the PATCH number.

7. Presentation and Justification

The solution is presented in a short deck to the instructor and classmates. The outline below shows what each slide says and why.

#	Slide	Key message
1	Problem	Outsiders must contribute without write access and without wasting maintainers' time
2	Stakeholders and goals	Contributor, maintainers, users and the community
3	The workflow (Diagram 1)	Read rules, fork, issue, branch, MR, review, squash, release notes
4	Fork and remotes (Diagram 2)	origin, upstream and keeping the fork current
5	Review loop (Diagram 3)	Feedback is a conversation; how to respond
6	Clean history (Diagram 4)	Squash before merge, and the force push safety rule
7	Release notes	Telling users what changed and who made it happen
8	Results and plan	Success metrics and a two-week schedule

Why this approach is justified

Risk (from Section 4)	How the workflow reduces it
Ignoring the rules (C1, C9)	Step 6.1 makes reading the rules, licence and sign-off requirements the first task
Wasted work (C2)	Agree on the change in an issue before writing code
Outdated fork (C3)	upstream remote and a sync routine before each branch
Big, slow reviews (C4)	One issue, one branch, small merge requests
Failing checks (C5)	Run tests and linters locally, and check the pipeline
Stalled reviews and tone (C6, C10)	Clear feedback routine, polite reminders and the code of conduct
Messy history (C7)	Squash into meaningful commits
Lost work from rewriting history (C8)	Rewrite only your own branch, using force-with-lease
Leaked secrets (C11)	Review the diff before each commit and push

Alternatives considered: committing directly to the project is not possible without write access; sending patches by email is still used by some projects but is harder to review and track. The fork and merge request model is the most common and gives a visible record of the discussion.

8. Implementation Plan

The simulation is planned for about **two weeks** (an assumption).

Phase	When	Activities	Output
1. Explore	Day 1 to 2	Choose the project; read README, CONTRIBUTING, code of conduct and licence; browse issues	Short notes on the rules
2. Fork and set up	Day 3	Fork, clone, add upstream, run the project and tests	Working local copy
3. Issue	Day 3 to 4	Search, raise the issue, discuss and get agreement	Issue #214 agreed
4. Implement	Day 4 to 7	Create the feature branch; code, test and document the change	Tested branch on the fork
5. Merge request	Day 8	Open the MR; fill the template; check the pipeline	MR !231 open
6. Review	Day 9 to 11	Answer feedback; push fixes; resolve threads	Approved MR
7. Squash and merge	Day 12	Squash commits; maintainer merges; clean up the fork	Change in upstream
8. Release notes	Day 13	Write the changelog entry; check the release	Release notes published
9. Review	Day 14	Retrospective and presentation	Lessons and report

Tools and preparation

- Git, a GitLab account with two-factor authentication, and a code editor.
- The project's build tools (for example Java and Maven) and its test and lint commands.
- A simple log (a notes file) of each step, with links to the issue and merge request as evidence.

9. Measurable Success Criteria

Targets are illustrative (Assumption A7). Real baselines should be measured and targets adjusted.

#	Metric	How measured	Target
S1	Contribution rules read and checklist completed	Notes file	100%
S2	Merge requests linked to an agreed issue	MR description	100%
S3	Size of the merge request	MR changes tab	About 300 lines or fewer
S4	Pipeline status at review time	MR pipeline panel	Passing
S5	Review feedback answered	Resolved threads	100% within 48 hours
S6	Rounds of review before approval	MR activity	3 or fewer
S7	Commits after squashing	git log	1 to 3 meaningful commits
S8	Force pushes to shared branches	Branch history	0
S9	Merge request accepted by a maintainer	MR status	Merged
S10	Release notes entry present and clear	Changelog or release page	Yes
S11	Code of conduct or licence problems raised	Maintainer comments	0

10. Recommendations and Lessons Learned

10.1 Recommendations

1. Read the contribution guide and code of conduct before doing anything else.
2. Raise or find an issue and agree on the change before writing code.
3. Keep your fork in sync with upstream and start every branch from the latest code.
4. Keep merge requests small, focused, tested and well described.
5. Treat review feedback as teamwork: respond quickly, politely and clearly.
6. Squash commits into a clean history, and force push only on your own feature branch.
7. Write release notes for users, with links to the issue and merge request.
8. Start with small contributions (documentation, tests, small fixes) to build trust.

10.2 Lessons Learned

These come from designing this solution, not from a live project.

- **Communication comes before code:** an issue discussion can save days of work.
- **Small changes are accepted faster** and are easier to review.
- **A fork gives freedom with safety:** you can experiment without affecting anyone.
- **History is documentation:** clean commits and release notes help future contributors and users.
- **Patience and respect matter:** maintainers are often volunteers, and kindness builds a good reputation.